LESSON FOUR

GIT COMMANDS AND USING THE TERMINAL TO SAVE COMMITS

4.1 Meaning of Git Commands

I learned that Git commands are instructions used to communicate with Git through a terminal or command-line interface. These commands help developers create repositories, track changes, save commits and upload projects to GitHub.

4.2 Important Git Commands

The following are some important Git commands I learned:

Git Command	Explanation
`git --version`	Checks the installed Git version.
`git init`	Initializes a new Git repository.
`git status`	Shows the current status of files.
`git add .`	Adds all changed files to the staging area.
`git add filename`	Adds a specific file.
`git commit -m "message"`	Saves changes with a message.
`git log`	Displays the commit history.
`git branch`	Displays available branches.
`git branch name`	Creates a new branch.
`git checkout name`	Switches to another branch.
`git switch name`	Switches to another branch.
`git merge name`	Merges another branch into the current branch.
`git clone URL`	Downloads a remote repository.
`git push`	Uploads local commits to GitHub.
`git pull`	Downloads and integrates remote changes.
`git fetch`	Downloads remote updates without integrating them.
`git remote -v`	Displays remote repository addresses.
`git diff`	Shows changes made to files.
`git rm filename`	Removes a tracked file.
`git restore filename`	Discards unstaged changes to a file.

4.3 Using the Terminal

I learned that a terminal is a command-line environment where I can enter instructions to perform different operations on my computer.

Instead of using only graphical interfaces, I can use the terminal to create folders, open project directories, initialize Git and manage my project.

Examples of common terminal commands include:

```
mkdir my-project
```

This creates a new folder called my-project.

```
cd my-project
```

This enters the project folder.

```
dir
```

This displays the files and folders in the current directory on Windows.

4.4 How to Save Commits Using the Terminal

I learned that saving commits is an important part of using Git. It allows me to record changes made to my project.

The following steps explain how I can save my work:

Step 1: Open the project folder

```
cd my-project
```

Step 2: Initialize Git

```
git init
```

Step 3: Check the project status

```
git status
```

Step 4: Add files to the staging area

```
git add .
```

Step 5: Save the changes as a commit

```
git commit -m "Initial project files"
```

Step 6: Connect the local repository to GitHub

```
git remote add origin https://github.com/username/my-project.git
```

Step 7: Push the project to GitHub

```
git push -u origin main
```

The GitHub repository URL and username should be replaced with the actual details of my repository.

4.5 Why Commit Messages Are Important

I learned that commit messages describe the changes saved in a repository. They help me and other developers understand the history of a project.

Examples of good commit messages include:

- Added login page.
- Updated homepage design.
- Fixed registration error.
- Created database connection.
- Added student dashboard.

Conclusion

From this lesson, I understood that Git commands help me manage my software projects through the terminal. I also learned how to initialize a repository, add files, save commits and push my work to GitHub.